

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Тема: Динамическое программирование

Студент гр. 4345

Наумов М. В.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

В стране проходят выборы президента. Боб пришёл на участок и должен вписать в бюллетень фамилию понравившегося кандидата. Он начал вписывать фамилию кандидата, но обнаружил, что написал фамилию не того кандидата. Сколько минимально исправлений должен сделать Боб, чтобы исправить запись с **ВАСИЛЬЕВ** на **КВАСЮТИН**? Боб может исправлять запись тремя способами:

1. Зачеркнуть любую букву и написать поверх неё другую (замена).
2. Вставить любую букву между двух других (вставка).
3. Зачеркнуть любую букву (удаление).

В ответе укажите только **одно** число — минимальное количество исправлений.

Вар. 4. Два подварианта:

4б. Добавляется 4-я операция со своей стоимостью: замена двух символов на один символ.

Описание алгоритма

Основная идея алгоритма заключается в использовании восходящего динамического программирования (bottom-up DP) для вычисления минимальной стоимости трансформации одной строки в другую с учетом расширенного набора операций.

В отличие от жадных подходов, алгоритм не делает локальных предположений. Задача разбивается на сетку пересекающихся подзадач, результат которых аккумулируется в двумерной матрице dp . Каждая ячейка $dp[i][j]$ хранит минимальную стоимость преобразования префикса строки A (длиной i) в префикс строки B (длиной j). В данной реализации значение в каждой клетке вычисляется на основе четырех возможных предыдущих состояний:

Удаление (переход из $dp[i-1][j]$).

Вставка (переход из $dp[i][j-1]$).

Замена/Совпадение (переход из $dp[i-1][j-1]$).

Специфическая операция 4б (замена двух символов строки A на один символ строки B , переход из $dp[i-2][j-1]$).

Благодаря системному перебору всех вариантов и выбору минимального из них ($\min$), гарантируется, что итоговый ответ в ячейке $dp[n][m]$ является глобально оптимальным.

Сложность по времени: $O(N * M)$

Основной цикл: Программа использует два вложенных цикла, проходящих по длинам строк N и M . Это дает ровно $N * M$ итераций основной логики.

Вспомогательные операции: Внутри циклов выполняются элементарные арифметические действия и сравнения. Проверка условия для операции 4б ($i \geq 2$) и нахождение минимума из четырех вариантов выполняются за константное время $O(1)$.

Логирование: Запись в файл log.txt происходит на каждой итерации. Поскольку объем записываемых данных для каждой ячейки фиксирован, это добавляет константный множитель, не меняя общую асимптотику $O(N * M)$.

Итоговая временная сложность: $O(N * M)$.

Сложность по памяти: $O(N * M)$

Матрица состояний: Основной объем памяти выделяется под двумерный массив `dp` размером $(N+1) * (M+1)$. Использование полной матрицы необходимо для обеспечения корректной работы алгоритма и возможности (при необходимости) восстановить полный путь трансформации.

Стек и переменные: Алгоритм реализован итеративно, что исключает риск переполнения стека (в отличие от рекурсивных версий). Дополнительные переменные для хранения стоимостей и текущих символов занимают пренебрежимо малое пространство $O(1)$.

Особенности и модификации алгоритма

Несмотря на использование классического каркаса матрицы Вагнера — Фишера, данная реализация включает ряд специфических особенностей, адаптированных под расширенные требования задачи:

1. Поддержка комплексной операции «замена двух на один» (4б):

Ключевым отличием данной модификации является внедрение четвертого типа перехода в динамическом программировании. Алгоритм рассматривает операцию замены двух символов исходной строки *A* на один символ целевой строки *B* с фиксированной стоимостью `cost_individ`. Математически это реализовано через обращение к состоянию `dp[i - 2][j - 1]`. Данная модификация позволяет алгоритму находить более эффективные пути трансформации в лингвистических сценариях (например, при обработке диграфов или специфических сокращений).

2. Гибкая настройка весовых коэффициентов:

В отличие от базового расстояния Левенштейна, где цена любой правки по умолчанию равна 1, программа оперирует набором независимых переменных: `cost_del`, `cost_ins`, `cost_repl` и `cost_individ`. Это превращает

алгоритм в универсальный инструмент, который можно настроить под конкретную задачу: от классического поиска опечаток до специфических задач биоинформатики или анализа текстов, где удаление может быть значительно «дороже» вставки.

3. Интегрированная система пошагового аудита (логирование):

В алгоритм встроен механизм детальной фиксации процесса вычислений в файл `log.txt`. Система регистрирует:

Состояние матрицы после этапа инициализации граничных условий.

Подробный расчет для каждой ячейки `dp[i][j]`, включая промежуточные суммы для всех четырех доступных операций.

Финальный вид заполненной матрицы.

Такой подход обеспечивает полную прозрачность работы «черного ящика» динамического программирования, позволяя визуально верифицировать корректность выбора минимальной стоимости в спорных ситуациях.

4. Использование принципа «бесконечности» для граничных условий:

При расчете операции 4б в строках, где $i < 2$, алгоритм использует значение `float('inf')`. Это элегантное программное решение позволяет сохранить единообразие логики функции `min()`, исключая невозможные переходы из процесса сравнения без загромождения кода сложными вложенными проверками.

Способ хранения промежуточных состояний

Для оптимизации процесса вычислений и последующего восстановления маршрута трансформации в данной реализации используется структура «двухслойного» хранения данных.

Матрица состояний: Вместо дерева рекурсивных вызовов используется итеративное заполнение двумерного массива (матрицы) `dp` размером $(N+1) * (M+1)$. В отличие от классической реализации, каждая ячейка матрицы представляет собой не просто число, а список (или кортеж) вида `[cost, operation]`.

cost — минимальная накопленная стоимость для данной подзадачи.

operation — строковый идентификатор операции ('M', 'R', 'D', 'I' или '4'), которая привела к этому минимуму.

Это позволяет избежать повторных сравнений стоимостей при восстановлении пути, так как оптимальное решение для каждой точки уже зафиксировано в момент расчета.

Восстановление транскрипта (бэктрекинг):

Для формирования итоговой последовательности операций используется строковая переменная *s*, работающая по принципу стека. Алгоритм начинает движение из финальной точки $dp[n][m]$ и перемещается к началу $dp[0][0]$, руководствуясь символом операции, записанным в текущей ячейке.

При встрече 'M' или 'R' индексы уменьшаются на (1, 1).

При встрече 'D' индекс строки *i* уменьшается на 1.

При встрече 'I' индекс столбца *j* уменьшается на 1.

При встрече '4' (операция 4б) совершается прыжок на две строки вверх и один столбец влево: $i -= 2, j -= 1$.

Формирование результата:

Поскольку обратный проход собирает символы от конца к началу, итоговая строка *s* подвергается процедуре реверса ($s[::-1]$). Это обеспечивает вывод хронологически верной последовательности действий, преобразующей строку *A* в строку *B*.

Описание итеративной функции и обратного хода (бэктрекинг)

Основа алгоритма — итеративная процедура заполнения матрицы и последующий восстановительный анализ (бэктрекинг). В отличие от рекурсивных подходов, данная реализация не использует стек вызовов для поиска пути «сверху вниз», а последовательно выстраивает фундамент решений «снизу вверх» (метод *tabulation*).

Итеративное заполнение (Прямой ход):

После инициализации нулевых строки и столбца (базовые случаи, где строка превращается в пустую и наоборот), алгоритм сканирует матрицу строго слева направо и сверху вниз с помощью двух вложенных циклов. В каждой ячейке программа рассчитывает стоимости четырех потенциальных действий:

Удаления (`cost_delete`);

Вставки (`cost_insert`);

Замены/Совпадения (`cost_replace`);

Операции 4б из индивидуального варианта (`cost_individual`).

Алгоритм выбирает строго минимальное из этих значений и сохраняет его в `dp[i][j]` вместе с маркером соответствующей операции. Это гарантирует, что каждый локальный шаг является частью глобально оптимального пути.

Восстановление пути (Бэктрекинг):

После завершения расчетов в игру вступает механизм бэктрекинга. Программа выполняет обратный проход по матрице, начиная от финального состояния `dp[n][m]`.

На каждом шаге алгоритм анализирует маркер операции, сохраненный в ячейке, и «отматывает» изменения назад. Благодаря тому, что на этапе прямого хода в каждой клетке был зафиксирован именно тот выбор, который привел к минимуму, бэктрекинг безошибочно восстанавливает кратчайшую последовательность правок. Этот процесс продолжается до тех пор, пока оба индекса (*i* и *j*) не станут равны нулю, что соответствует полному возврату к исходному состоянию строк. Результат бэктрекинга инвертируется, формируя итоговый транскрипт.

Структуры данных и ключевые переменные:

A и B (строки) — исходная и целевая последовательности символов, считанные с помощью `input().strip()`. Их длины обозначаются как n и m .

`dp` (двумерный список) — матрица состояний размером $(n+1) \times (m+1)$. Каждая ячейка `dp[i][j]` содержит список `[cost, op]`, где `cost` —

минимальная стоимость трансформации префиксов, а `op` — символ операции ('M', 'R', 'D', 'I' или '4'), приведшей к этому результату. Это позволяет избежать повторных вычислений и упрощает бэктрекинг.

`s` (строка) — переменная, аккумулирующая символы операций при восстановлении пути. Работает по принципу стека: символы добавляются в конец, а затем вся строка разворачивается.

`cost_repl`, `cost_ins`, `cost_del`, `cost_individ` (целые числа) — весовые коэффициенты (штрафы) для базовых операций и специфической операции «замена двух на один».

Функции и логические блоки

`log(text)` и `log_matrix()` — вложенные вспомогательные функции для аудита. Они записывают этапы вычислений в файл `log.txt`. `log_matrix` визуализирует текущее состояние числовых значений в матрице `dp`, добавляя заголовки из символов строк `A` и `B` для наглядности.

`solve()` — основная функция, инкапсулирующая всю логику алгоритма: от ввода данных до вывода результата.

Этапы работы алгоритма:

Инициализация (Базовые случаи):

С помощью двух независимых циклов заполняются нулевой столбец и нулевая строка матрицы `dp`. Каждая ячейка получает накопленную стоимость (только удаления для столбца и только вставки для строки) и соответствующий маркер операции ('D' или 'I').

Заполнение основной таблицы (Прямой ход):

Двойной цикл сканирует матрицу. Для каждой клетки `dp[i][j]` вычисляются стоимости четырех вариантов:

`cost_delete` (сверху + цена удаления);

`cost_insert` (слева + цена вставки);

`cost_replace` (по диагонали + цена замены или 0 при совпадении);

`cost_individual` (через одну строку выше + цена операции 4б, если $i \geq 2$).

В ячейку записывается строго минимальное значение и буква операции. Приоритет выбора (при равных стоимостях) определяется последовательностью if-elif.

Бэктрекинг (Восстановление пути):

Цикл while начинает движение из ячейки `dp[n][m]` к `dp[0][0]`. Вместо пересчета стоимостей алгоритм просто считывает готовый маркер операции из `dp[i][j][1]`:

'M' / 'R': сдвиг на $(-1, -1)$;

'D': сдвиг на $(-1, 0)$;

'T': сдвиг на $(0, -1)$;

'4': сдвиг на $(-2, -1)$.

Символы добавляются в строку `s`, которая в конце инвертируется (`s[::-1]`).

Главный блок программы (`if __name__ == "__main__"`)

Управляющая логика, которая запускает функцию `solve()`. Она обеспечивает последовательное выполнение: чтение параметров, запуск вычислений, вывод итогового расстояния в консоль и создание файла лога для детального анализа работы алгоритма.

Тестирование

Результаты тестирования представлены в табл. 1.

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные	Комментарии
1.	1 1 1 1 ВАСИЛЬЕВ КВАСЮТИН	5 IMMM44RI	Верный результат
2.	20 30 40 90 СТРОКА СТРОКА	0 MMMMMM	При вводе одинаковых строк программа действует верно
3.	2 3 2 2 СЛОВО	15 III	Вводом пустой строки программу тоже не “обмануть”
4.	3 4 3 4 СЛОВСОЧЕТАНИЕ ВОЛОСОК	26 4MRMMM4DDDDD	Работа с примерами посложнее

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.py

```
def solve():
    costs = list(map(int, input().split()))
    cost_repl, cost_ins, cost_del, cost_individ = costs
    A = input().strip()
    B = input().strip()

    n, m = len(A), len(B)
    dp = []
    for i in range(n + 1):
        dp.append([])
        for j in range(m + 1):
            dp[i].append([0, ''])

    with open("log.txt", "w", encoding="utf-8") as log_file:
        def log(text):
            log_file.write(text + "\n")

        def log_matrix():
            header = "      " + " ".join(f"{ch:>3}" for ch in B)
            log(header)
            for r in range(n + 1):
                row_char = " " if r == 0 else A[r - 1]
                row_vals = " ".join(f"{dp[r][c][0]:>3}" for c in
range(m + 1))
                log(f"{row_char} {row_vals}")
            log("")

        log(f"Начало вычисления.\nСтрока A: '{A}'\nСтрока B: '{B}'\n")

        for i in range(1, n + 1):
            dp[i][0] = [dp[i - 1][0][0] + cost_del, 'D']

        for j in range(1, m + 1):
            dp[0][j] = [dp[0][j - 1][0] + cost_ins, 'I']

        log("Матрица после инициализации краев:")
        log_matrix()

        for i in range(1, n + 1):
            for j in range(1, m + 1):
                cost_delete = dp[i - 1][j][0] + cost_del
                cost_insert = dp[i][j - 1][0] + cost_ins

                if A[i - 1] == B[j - 1]:
                    cost_replace = dp[i - 1][j - 1][0]
                    rep_type = "Совпадение"
                else:
                    cost_replace = dp[i - 1][j - 1][0] + cost_repl
                    rep_type = "Замена"

                if i >= 2:
```

```

        cost_individual = dp[i - 2][j - 1][0] + cost_in-
divid
        can_4b = True
    else:
        cost_individual = float('inf')
        can_4b = False

    mn = min(cost_delete, cost_insert, cost_replace,
cost_individual)
    if cost_delete == mn:
        dp[i][j] = [mn, 'D']
    elif cost_insert == mn:
        dp[i][j] = [mn, 'I']
    elif cost_replace == mn:
        if A[i - 1] == B[j - 1]:
            dp[i][j] = [mn, 'M']
        else:
            dp[i][j] = [mn, 'R']
    else:
        dp[i][j] = [mn, '4']

    log(f"Клетка [{i}][{j}] | A[{i - 1}]='{A[i - 1]]',
B[{j - 1}]='{B[j - 1]]'"")
    log(f" 1. Удаление: {dp[i - 1][j][0]} + {cost_del} =
{cost_delete}")
    log(f" 2. Вставка: {dp[i][j - 1][0]} + {cost_ins} =
{cost_insert}")
    log(f" 3. {rep_type}: {dp[i - 1][j - 1][0]} + {0 if
rep_type == 'Совпадение' else cost_repl} = {cost_replace}")
    if can_4b:
        log(f" 4. Операция из индивидуального варианта:
{dp[i - 2][j - 1][0]} + {cost_individ} = {cost_individual}")
    log(f" Результат dp[{i}][{j}] = {dp[i][j][0]}\n")

    log("Финальная матрица:")
    log_matrix()
    log(f"Итоговое расстояние: {dp[n][m][0]}")

print(dp[n][m][0])
i = n
j = m
s = ''
while i != 0 or j != 0:
    s += dp[i][j][1]
    if s[-1] == 'M' or s[-1] == 'R':
        i -= 1
        j -= 1
    elif s[-1] == 'D':
        i -= 1
    elif s[-1] == 'I':
        j -= 1
    else:
        i -= 2
        j -= 1

s = s[::-1]
print(s)

```

```
if __name__ == "__main__":  
    solve()
```